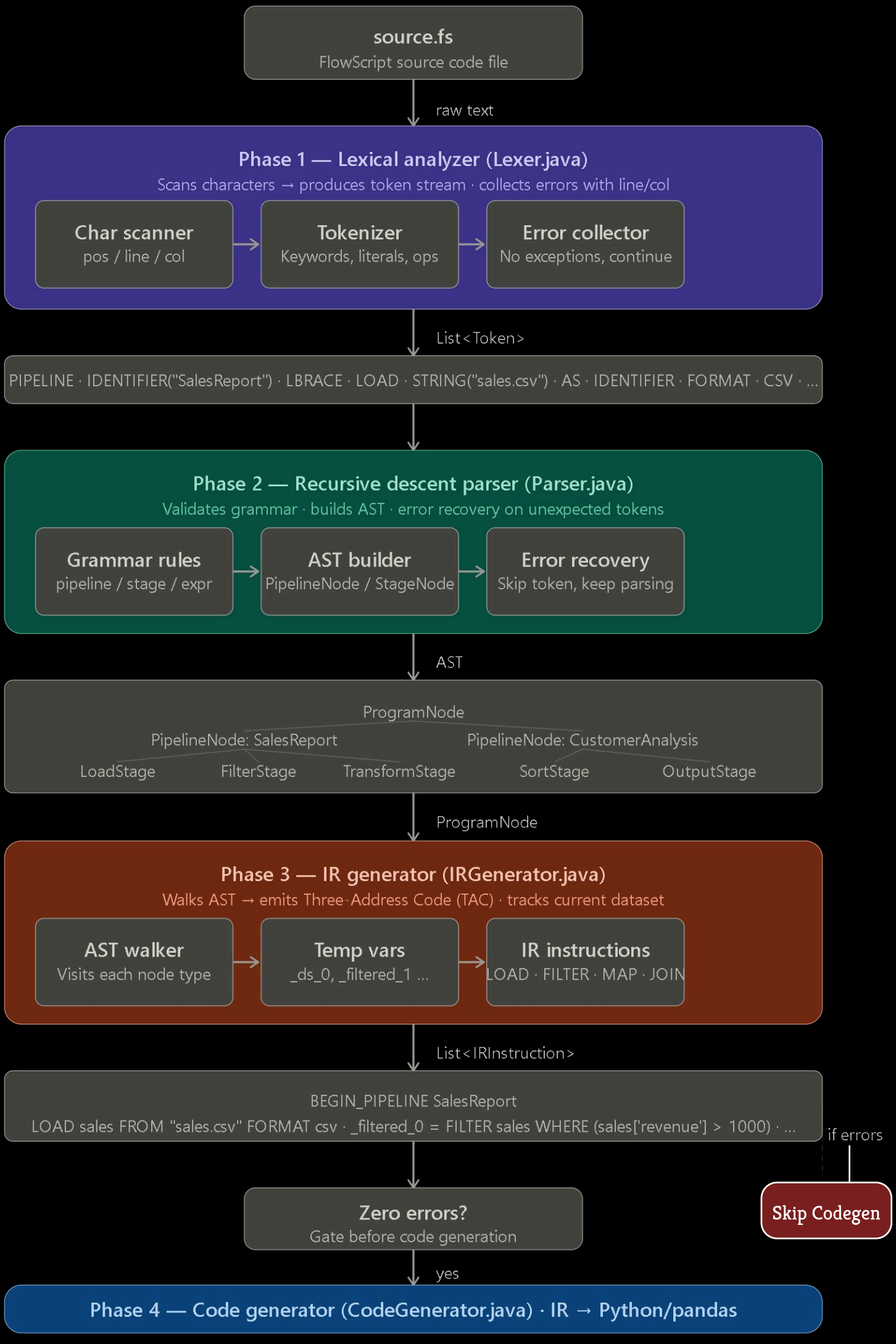

source.fs
FlowScript source code file
raw text
Phase 1 — Lexical analyzer (Lexer.java)
Scans characters → produces token stream · collects errors with line/col
Char scanner
pos / line / col
Tokenizer
Keywords, literals, ops
Error collector
No exceptions, continue
List<Token>
PIPELINE · IDENTIFIER("SalesReport") · LBRACE · LOAD · STRING("sales.csv") · AS · IDENTIFIER · FORMAT · CSV · ...
Phase 2 — Recursive descent parser (Parser.java)
Validates grammar · builds AST · error recovery on unexpected tokens
Grammar rules
pipeline / stage / expr
AST builder
PipelineNode / StageNode
Error recovery
Skip token, keep parsing
AST
ProgramNode
PipelineNode: SalesReport
PipelineNode: CustomerAnalysis
LoadStage
FilterStage
TransformStage
SortStage
OutputStage
ProgramNode
Phase 3 — IR generator (IRGenerator.java)
Walks AST → emits Three-Address Code (TAC) · tracks current dataset
AST walker
Visits each node type
Temp vars
_ds_0, _filtered_1 ...
IR instructions
LOAD · FILTER · MAP · JOIN
List<IRInstruction>
BEGIN_PIPELINE SalesReport
LOAD sales FROM "sales.csv" FORMAT csv · _filtered_0 = FILTER sales WHERE (sales['revenue'] > 1000) · ...
if errors
Skip Codegen
Zero errors?
Gate before code generation
yes
Phase 4 — Code generator (CodeGenerator.java) · IR → Python/pandas